



VIRTUAL KEYBOARD

**PROJECT
REPORT**
JUNE
2025

MINI PROJECT REPORT SUBMITTED IN PARTIAL
FULFILLMENT OF THE REQUIREMENTS FOR THE
AWARD OF THE DEGREE OF **BACHELOR OF
TECHNOLOGY** IN INFORMATION TECHNOLOGY OF
THE ANNA UNIVERSITY

**PROJECT
WORK**

Submitted by
VEERAKUMAR D - 722824205060

BATCH
2024 - 2028

Under the Guidance of
Mr. G. SWAMINATHAN M.Tech.,(PhD).,
ASSISTANT PROFESSOR

INFORMATION TECHNOLOGY
Sri Eshwar College of Engineering
(An Autonomous Institution – Affiliated to Anna University)
COIMBATORE – 641 202

Sri Eshwar College of Engineering
(An Autonomous Institution – Affiliated to Anna University)
COIMBATORE – 641 202

BONAFIDE CERTIFICATE

Certified that this Report titled “**VIRTUAL KEYBOARD**” is the Bonafide work of
VEERAKUMAR D – 24IT060 who carried out the project work under my
supervision.

SIGNATURE

Dr. R. SIAMALA DEVI M.E., Ph.D.,
PROFESSOR

HEAD OF THE DEPARTMENT

INFORMATION TECHNOLOGY

Sri Eshwar College of Engineering,
Coimbatore – 641 202.

SIGNATURE

Mr. G. SWAMINATHAN M.Tech.,(PhD),
SUPERVISOR

ASSISTANT PROFESSOR

INFORMATION TECHNOLOGY

Sri Eshwar College of Engineering,
Coimbatore – 641 202.

Submitted for the **Autonomous Semester End Mini Project Viva-Voce** held on

.....

INTERNAL EXAMINER

EXTERNAL EXAMINER

	TABLE OF CONTENTS		
CHAPTER NO.	TITLE		PAGE NO.
	ABSTRACT		i
	LIST OF FIGURES		ii
1	INTRODUCTION		
	1.1	OBJECTIVES	1
	1.2	SCOPE OF THE PROJECT	1
2	SYSTEM ANALYSIS AND SPECIFICATION		
	2.1	PROBLEM DESCRIPTION	2
	2.2	FUNCTIONAL REQUIREMENT HARDWARE & SOFTWARE	2
	2.3	NON – FUNCTIONAL REQUIREMENT	3
3	PROJECT DESCRIPTION		
	3.1	MODULE DESCRIPTION	4
4	IMPLEMENTATION		5
5	RESULTS AND DISCUSSION		12
6	CONCLUSION & FUTURE ENHANCEMENT		14

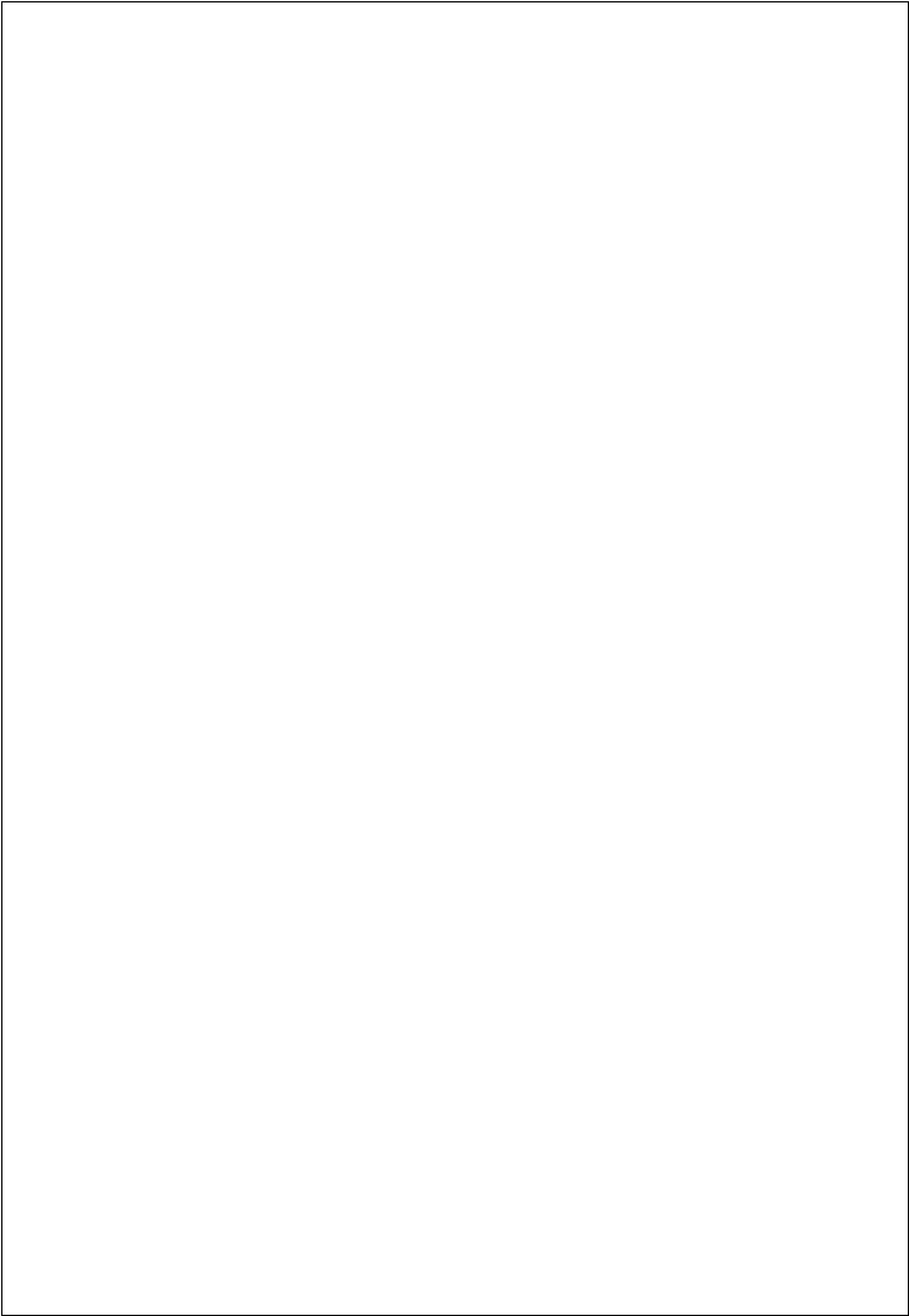
ABSTRACT

The AI Virtual Keyboard is an innovative solution designed to enhance human-computer interaction by allowing users to type without a physical keyboard. Leveraging computer vision techniques and OpenCV, the system detects finger or hand movements via a webcam, enabling users to interact with a virtual keyboard displayed on the screen. This contactless input method provides an accessible and hygienic alternative for individuals with mobility challenges or in environments where touching surfaces is undesirable, such as hospitals or public kiosks. The system accurately tracks finger positions, highlights the selected keys, and confirms input when the user gestures a "press."

In addition to its core functionality, the project is integrated with a MongoDB database to store each input entered by the user when the "Enter" key is pressed. This feature adds a layer of data persistence and can be used for monitoring user input, performing analytics, or enabling custom features like input history. The use of FastAPI as the backend allows efficient communication between the virtual keyboard and the database, ensuring real-time data handling and scalability. This project demonstrates the potential of AI and computer vision in creating interactive, touchless interfaces that are both practical and forward-thinking.

LIST OF FIGURES

FIG.NO	TITLE	PAGE NO.
5.1	LANDING MODULE	12
5.2	HOME PAGE	12
5.3	BLOG PAGE	12
5.4	ABOUT MODULE	13
5.5	PROFILE PAGE	13



CHAPTER 1

INTRODUCTION

1.1 OBJECTIVES

The main objective of this project is to create a virtual keyboard that operates without physical contact using computer vision and OpenCV. It aims to capture real-time finger movements through a webcam and translate them into key presses on a virtual interface. The system is designed to improve accessibility for users with physical disabilities and reduce the spread of germs by eliminating the need to touch a physical keyboard. The project also focuses on providing a smooth, responsive, and accurate typing experience through precise fingertip detection and gesture recognition.

Additionally, the project aims to integrate FastAPI to establish seamless backend communication. Each time the user presses the "Enter" key, their input is stored in a MongoDB database for future reference or analysis. This feature supports data logging, monitoring, and potential enhancements like predictive text or usage insights. The objective also includes ensuring real-time input handling, system stability, and demonstrating the practical use of AI in human-computer interaction. The end goal is to build an innovative, functional, and user-friendly touchless typing system.

1.2 SCOPE OF THE PROJECT

The AI Virtual Keyboard project explores the integration of computer vision, artificial intelligence, and modern backend technologies to create a touchless typing solution. It is designed to operate entirely through a webcam, detecting finger movements and gestures to simulate key presses on a virtual keyboard. This makes it particularly useful in environments where hygiene and contactless interaction are priorities, such as hospitals, public kiosks, and industrial settings. It also enhances accessibility for users with physical impairments who may find traditional keyboards difficult to use.

The scope of the project extends beyond basic typing functionality. It includes real-time fingertip tracking, gesture-based input recognition, and integration with a backend system using FastAPI. Additionally, user inputs are stored in a MongoDB database each time the "Enter" key is pressed, allowing for input history, analytics, or future features like predictive suggestions. The project demonstrates a scalable, customizable solution that can be enhanced with features like multilingual support, voice input, or cloud-based synchronization in future versions.

CHAPTER 2

SYSTEM ANALYSIS AND SPECIFICATION

2.1 PROBLEM DESCRIPTION

Traditional physical keyboards require direct contact, which can be problematic in shared or public environments where hygiene is a concern. They also pose challenges for individuals with physical disabilities who may struggle with standard typing interfaces. As digital interaction becomes more widespread, the need for alternative input methods is growing.

In many healthcare and industrial settings, contactless interaction is essential to reduce the risk of contamination. Touch-based systems can contribute to the spread of germs and are not always practical in sterile or hazardous environments. Existing alternatives like voice input may not be suitable in noisy or privacy-sensitive conditions.

There is a clear need for an accessible, hygienic, and efficient method of input that works without physical contact. A virtual keyboard powered by AI and computer vision offers a promising solution. By detecting finger movements via a camera, users can interact naturally and safely with digital devices without the need for touch.

2.2 FUNCTIONAL REQUIREMENT

CAMERA INPUT HANDLING

- The system must capture real-time video using the device's webcam.
- It should continuously process frames to detect hand and finger positions.

FINGERTIP TRACKING AND GESTURE DETECTION

- The system must accurately detect fingertip locations.
- It should recognize specific gestures to identify key presses.

VIRTUAL KEYBOARD DISPLAY

- A virtual keyboard must be rendered on the screen with clearly visible keys.
- The system should highlight keys when they are being pointed at.

TEXT INPUT PROCESSING

- Detected key presses must be converted into text and displayed in real-time.
- The system should support basic typing functions like space, backspace, and enter.

DATA STORAGE INTEGRATION

- Upon pressing the "Enter" key, the system must send the typed text to a backend API.
- The backend must store the input text in a MongoDB database with a timestamp.

BACKEND COMMUNICATION

- The frontend should communicate with the backend using HTTP POST requests.
- The system must confirm that the input was successfully stored.

ERROR HANDLING AND FEEDBACK

- The system must notify the user in case of gesture misdetection or camera failure.
- Feedback should be provided visually (e.g., highlighting or blinking key).

SOFTWARE AND HARDWARE REQUIREMENTS

SOFTWARE REQUIREMENTS

1. Operating System:

- Windows 10 or above / Linux / macOS

2. Programming Language:

- Python 3.8 or above

3. Libraries & Frameworks:

- OpenCV (for computer vision)
- NumPy (for array operations)
- FastAPI (for backend API)
- Requests (for API communication)
- Pymongo (for MongoDB integration)

4. Database:

- MongoDB (local or cloud-based like MongoDB Atlas)

5. Development Tools:

- Visual Studio Code / PyCharm / Jupyter Notebook
- Postman (for API testing)
- Git (for version control)

HARDWARE REQUIREMENT

1. Processor:

- Intel i5 or equivalent (minimum)
- Intel i7 or higher (recommended for real-time processing)

2. RAM:

- Minimum 4 GB
- Recommended 8 GB or more for smoother performance

3. Camera:

- In-built or external webcam (720p or higher resolution)

4. Storage:

- Minimum 500 MB free space for project files and dependencies

5. Display:

- Standard monitor (15" or above) for better UI visibility

NON-FUNCTIONAL REQUIREMENT

1. PERFORMANCE

- The system should process webcam input and detect finger gestures in real-time with minimal latency.
- Key press recognition and text rendering should occur within 1 second.

2. REABILITY

- The application must consistently detect finger positions without frequent errors or crashes.

- The backend should reliably store user inputs in the database every time the "Enter" key is pressed.

3. USABILITY

- The interface should be intuitive and user-friendly for both technical and non-technical users.
- Visual cues like key highlighting and on-screen feedback should guide users through interaction.

4. SCALABILITY

- The system should be able to handle multiple input sessions without performance degradation.
- Backend APIs and database should support future expansion, such as user accounts or advanced analytics.

5. PORTABILITY

- The application should run on various platforms including Windows, Linux, and macOS.
- It should require minimal configuration for setup on a new system.

6. SECURITY

- Data sent from the frontend to the backend should be secured against unauthorized access.
- Sensitive data (if extended to include login or user tracking) must be protected with encryption.

7. MAINTAINABILITY

- The code should be modular and well-documented to allow future updates or debugging.
- Components should be loosely coupled for easier maintenance and upgrades.

CHAPTER 3

PROJECT DESCRIPTION

3.1 MODULE DESCRIPTION

CAMERA INPUT & DETECTION MODULE

This module captures real-time video feed using a webcam. It processes the frames to detect and track the user's fingers or hand gestures using computer vision techniques provided by OpenCV.

VIRTUAL KEYBOARD INTERFACE MODULE

This module is responsible for displaying the on-screen keyboard layout. It highlights keys as the user points to them and detects the selection of a key when a specific gesture or press motion is made.

TEXT ASSEMBLY & DISPLAY MODULE

It collects the detected key presses and assembles them into meaningful text. The typed input is shown on-screen in real time to provide feedback to the user.

BACKEND COMMUNICATION MODULE

This module handles sending the completed input (when "Enter" is pressed) to the backend server using an API. It ensures smooth communication between the frontend (OpenCV interface) and the FastAPI backend.

DATABASE STORAGE MODULE

Using MongoDB, this module stores each user input with a timestamp. It allows for data persistence, retrieval, and potential analysis of user behavior or typing patterns.

ERROR HANDLING & FEEDBACK MODULE

This module ensures smooth user experience by handling gesture misinterpretation, missed key presses, and other potential issues. It provides feedback or prompts to help guide user input.

CHAPTER 4

IMPLEMENTATION

Home.jsx

```
<!-- Hero Section -->
<section class="relative pt-24 overflow-hidden" style="background-image:
url('https://readdy.ai/api/search-
image?query=A%20futuristic%20technology%20scene%20with%20holographic%20ke
yboard%20floating%20in%20the%20air%2C%20persons%20hands%20typing%20on%
20virtual%20keyboard%20with%20blue%20and%20purple%20light%20effects%2C%2
0minimalist%20clean%20background%20on%20the%20left%20side%20for%20text%2
0placement%2C%20modern%20tech%20aesthetic%2C%20high-
quality%20digital%20art&width=1920&height=1080&seq=1&orientation=landscape');
background-size: cover; background-position: center right;">
<div class="absolute inset-0 bg-gradient-to-r from-gray-900 via-gray-900/90 to-
transparent"></div>
<div class="container mx-auto px-6 py-32 md:py-40 relative z-10">
<div class="w-full max-w-2xl">
<h1 class="text-4xl md:text-5xl lg:text-6xl font-bold text-white leading-tight mb-
6">Type Without Touch.</h1>
<p class="text-xl md:text-2xl text-gray-200 mb-8">Gesture, speak, or predict your way
through words—powered by AI.</p>
<div class="flex flex-col sm:flex-row gap-4">
<a href="#demo" class="px-8 py-3 bg-primary text-white text-lg font-medium
!rounded-button whitespace-nowrap hover:bg-opacity-90 transition-all flex items-center
justify-center">
<span>Try It Now</span>
<div class="w-6 h-6 ml-2 flex items-center justify-center">
<i class="ri-arrow-right-line"></i>
</div>
</a>
<a href="#video" class="px-8 py-3 bg-white bg-opacity-10 backdrop-blur-sm text-white
border border-white border-opacity-20 text-lg font-medium !rounded-button
whitespace-nowrap hover:bg-opacity-20 transition-all flex items-center justify-center">
<div class="w-6 h-6 mr-2 flex items-center justify-center">
<i class="ri-play-circle-line"></i>
</div>
<span>Watch Demo</span>
</a>
</div>
</div>
```

```

    </div>
  </section>
  <!-- Features Section -->
  <section id="features" class="py-20 bg-white">
    <div class="container mx-auto px-6">
<div class="text-center mb-16">

```

Blog.jsx

```

<h2 class="text-3xl md:text-4xl font-bold text-gray-900 mb-4">Revolutionary Features</h2>
<p class="text-xl text-gray-600 max-w-3xl mx-auto">Experience the future of typing with our
AI-powered virtual keyboard that adapts to your needs.</p>
</div>
<div class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-8">
<!-- Feature 1 -->
<div class="feature-card bg-white p-8 rounded-lg shadow-lg border border-gray-100">
<div class="w-14 h-14 bg-primary/10 rounded-lg flex items-center justify-center mb-6">
<i class="ri-hand-line ri-2x text-primary"></i>
</div>
<h3 class="text-xl font-semibold text-gray-900 mb-3">Gesture Recognition</h3>
<p class="text-gray-600">Type in the air with natural hand movements. Our advanced camera
tracking detects precise finger positions.</p>
</div>
<!-- Feature 2 -->
<div class="feature-card bg-white p-8 rounded-lg shadow-lg border border-gray-100">
<div class="w-14 h-14 bg-primary/10 rounded-lg flex items-center justify-center mb-6">
<i class="ri-mic-line ri-2x text-primary"></i>
</div>
<h3 class="text-xl font-semibold text-gray-900 mb-3">Voice Input</h3>
<p class="text-gray-600">Seamlessly switch to voice typing when needed. Our AI understands
context and natural speech patterns.</p>
</div>
<!-- Feature 3 -->
<div class="feature-card bg-white p-8 rounded-lg shadow-lg border border-gray-100">
<div class="w-14 h-14 bg-primary/10 rounded-lg flex items-center justify-center mb-6">
<i class="ri-brain-line ri-2x text-primary"></i>
</div>
<h3 class="text-xl font-semibold text-gray-900 mb-3">AI Word Prediction</h3>
<p class="text-gray-600">Intelligent prediction learns from your writing style to suggest the
next words before you type them.</p>
</div>
<!-- Feature 4 -->
<div class="feature-card bg-white p-8 rounded-lg shadow-lg border border-gray-100">
<div class="w-14 h-14 bg-primary/10 rounded-lg flex items-center justify-center mb-6">
<i class="ri-keyboard-line ri-2x text-primary"></i>

```

```

</div>
<h3 class="text-xl font-semibold text-gray-900 mb-3">Multiple Layouts</h3>
<p class="text-gray-600">Switch between QWERTY, numeric, and symbol layouts with
simple gestures for complete typing flexibility.</p>
</div>
<!-- Feature 5 -->
<div class="feature-card bg-white p-8 rounded-lg shadow-lg border border-gray-100">
<div class="w-14 h-14 bg-primary/10 rounded-lg flex items-center justify-center mb-6">
<i class="ri-volume-up-line ri-2x text-primary"></i>
</div>
<h3 class="text-xl font-semibold text-gray-900 mb-3">Sound Feedback</h3>
<p class="text-gray-600">Customizable audio cues provide real-time feedback, confirming
your inputs without looking at the screen.</p>
</div>
<!-- Feature 6 -->
<div class="feature-card bg-white p-8 rounded-lg shadow-lg border border-gray-100">
<div class="w-14 h-14 bg-primary/10 rounded-lg flex items-center justify-center mb-6">
<i class="ri-edit-line ri-2x text-primary"></i>
</div>
<h3 class="text-xl font-semibold text-gray-900 mb-3">Smart Autocorrect</h3>
<p class="text-gray-600">Context-aware correction that understands the meaning of your
sentences to fix errors intelligently.</p>
</div>
</div>
</div>
</section>
<!-- Demo Section -->
<section id="demo" class="py-20 bg-gray-50">
<div class="container mx-auto px-6">
<div class="text-center mb-16">

```

index.js

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-
width, initial-scale=1.0" />
  <title>Document</title>
  <link
href='https://unpkg.com/boxicons@2.1.4/css/boxicon
s.min.css' rel='stylesheet'>
  <link rel="stylesheet" href="style.css" />

```

```

</head>
<body>
  <div class="container">
<!-- login form -->
    <div class="form-box login">
      <form action="">
        <h1>login</h1>
        <div class="input-box">
          <input type="text" placeholder="Username"
            required>
          <i class='bx bxs-user'></i>
        </div>
        <div class="input-box">
          <input
            type="password"
placeholder="password"
            required>
          <i class='bx bxs-lock-alt'></i>
        </div>
        <div class="forgot-link">
          <a href="#">forgot password?</a>
        </div>
        <button
          type="submit"
class="btn">login</button>
        <p>or login with social platforms</p>
        <div class="social-icons">
          <a href="#"><i class='bx bxl-google'
></i></a>
          <a href="#"><i class='bx bxl-facebook'
></i></a>
          <a href="#"><i class='bx bxl-github'
></i></a>
          <a href="#"><i class='bx bxl-linkedin'
></i></a>
        </div>
      </form>
    </div>
<!--Registration-->
    <div class="form-box register">
      <form action="">
        <h1>Registration</h1>
        <div class="input-box">
          <input type="text" placeholder="Username"
            required>
          <i class='bx bxs-user'></i>
        </div>

```



```

<div class="input-box">
  <input type="text" placeholder="Email"
    required>
  <i class='bx bxs-envelope'></i>
</div>
<div class="input-box">
  <input
    type="password"
placeholder="password"
    required>
  <i class='bx bxs-lock-alt'></i>
</div>
<button
    type="submit"
class="btn">Register</button>
<p>or Register with social platforms</p>
<div class="social-icons">
  <a href="#"><i class='bx bxl-google'
></i></a>
  <a href="#"><i class='bx bxl-facebook'
></i></a>
  <a href="#"><i class='bx bxl-github'
></i></a>
  <a href="#"><i class='bx bxl-linkedin'
></i></a>
</div>
</form>
</div>

```

```

<!--toggle-->
<div class="toggle-box">
  <div class="toggle-panel toggle-left">
    <h1>Hello, Welcome!</h1>
    <p>Don't have an account?</p>
    <button
      class="btn
register-
btn">Register</button>
  </div>
  <div class="toggle-panel toggle-right">
    <h1>Welcome Back!</h1>
    <p>Already have an account?</p>
    <button class="btn login-btn">Login</button>
  </div>
</div>

</div>

```

```
<script src="script.js"></script>  
</body>  
</html>
```

CHAPTER 5

RESULTS AND DISCUSSIONS

LOGIN PAGE: Handles user signup and login using username and password.

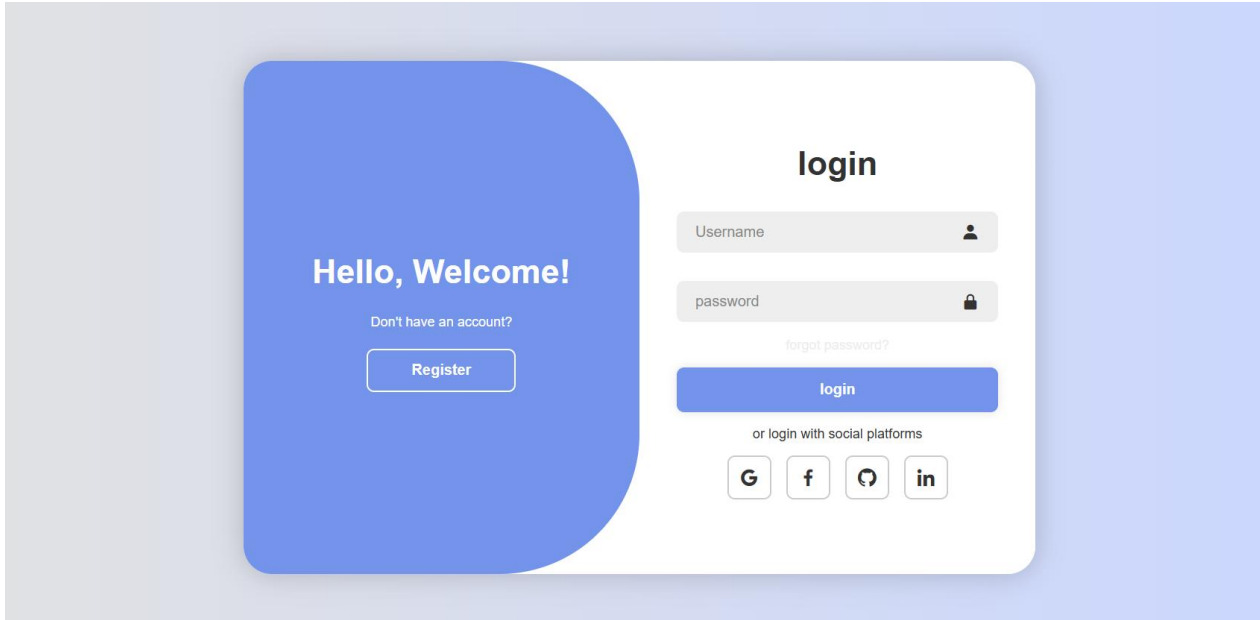


Fig:5.1 Landing Module

HOME: This landing page introduces Virtual Keyboard, an AI-powered tool that lets users type without touching a keyboard—using gestures, speech, or predictive input.

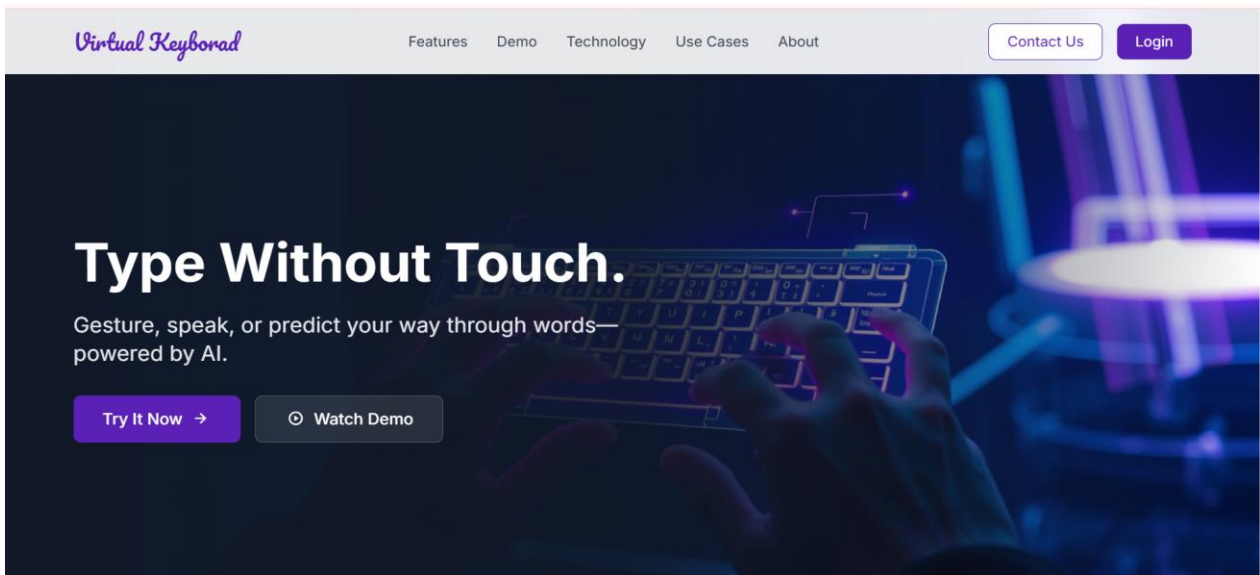


Fig:5.2 Home Page

BLOG: Revolutionary AI-powered virtual keyboard features touchless typing with gestures, voice input, prediction, and smart feedback.

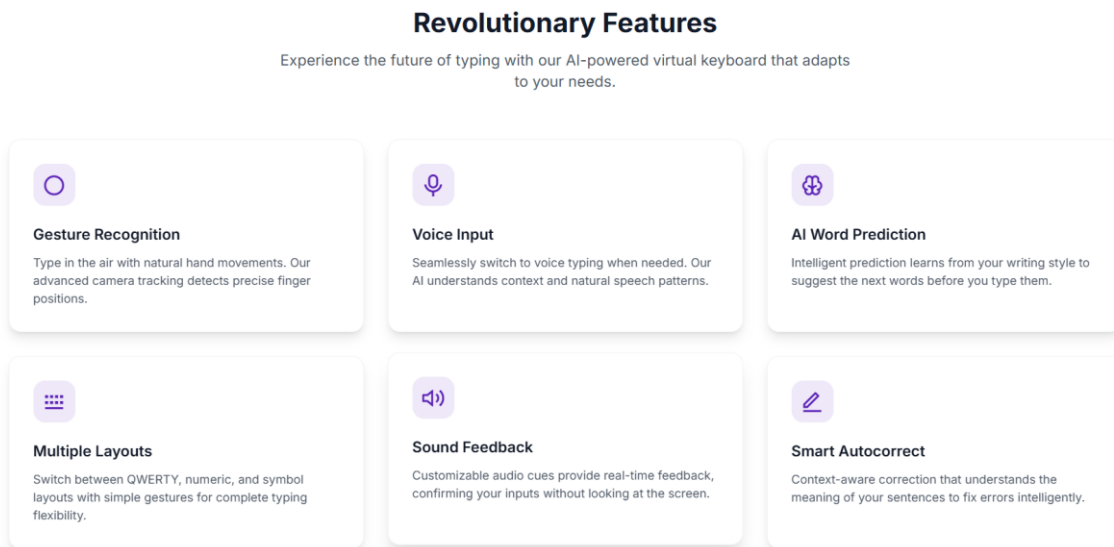


Fig:5.3 Blog Page

ABOUT: This page highlights AirType’s mission to revolutionize human-computer interaction by creating inclusive, adaptive, and cutting-edge virtual typing technology.

Our Mission

We're reimagining human-computer interaction by breaking free from physical constraints. AirType is more than a virtual keyboard—it's a step toward a future where technology adapts to humans, not the other way around.

Our team of engineers, designers, and accessibility experts are committed to creating technology that's both cutting-edge and inclusive.

[Join Our Beta](#)

[Read Our Story](#)



Fig:5.4 About Module

PROFILE: This page provides a contact form and essential contact details for users to reach out to AirType with questions, feedback, or inquiries.

Get in Touch

Have questions or feedback? We'd love to hear from you.

Contact Information

✉ info@airtype.tech

☎ +1 (555) 123-4567

📍 123 Innovation Drive
Colimbatore, CA 94107

Follow Us

X in y z

Name

Your name

Email

your.email@example.com

Subject

How can we help?

Message

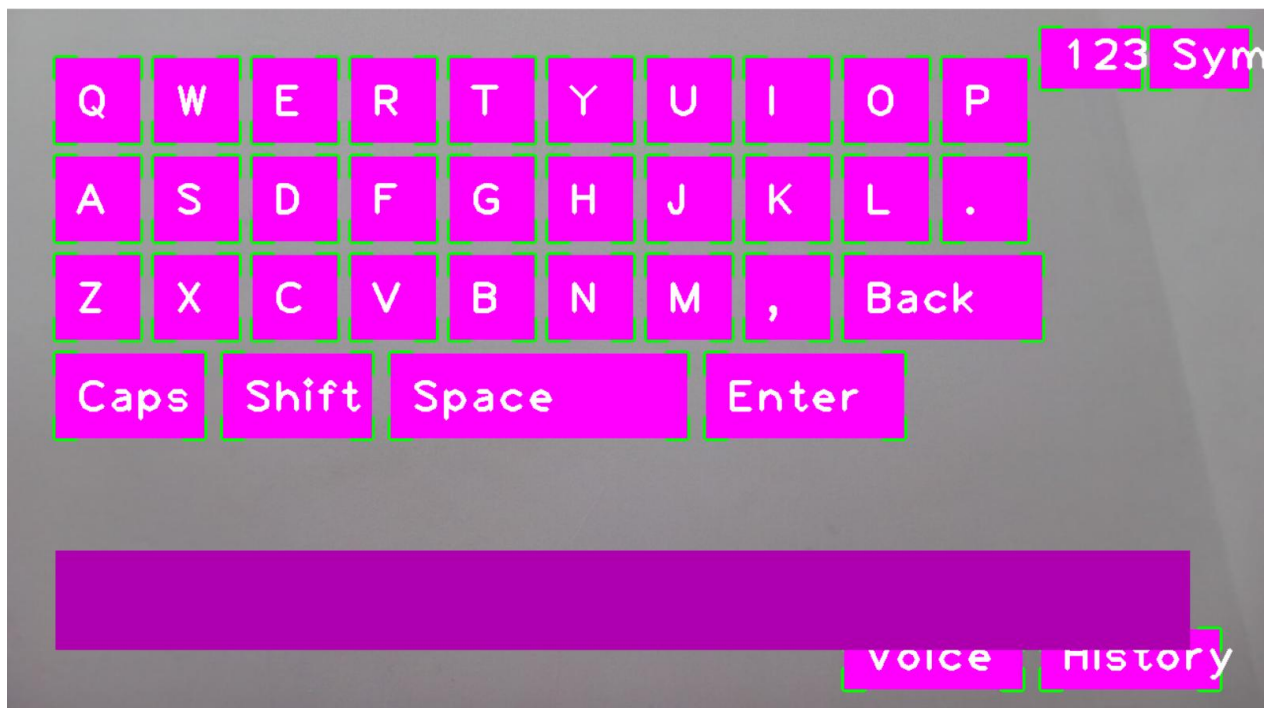
Tell us more about your inquiry...

☐ Subscribe to our newsletter for updates

Send Message

Fig:5.5 Profile Page

OUTPUT:



CHAPTER 6

CONCLUSION AND FUTURE ENHANCEMENT

The AI Virtual Keyboard project demonstrates the practical integration of computer vision and artificial intelligence to create an innovative, touchless typing interface. By leveraging real-time video input through a webcam and processing it using OpenCV, the system successfully tracks fingertip movements to simulate keyboard interaction without any physical contact. This provides a more hygienic and accessible method of input, particularly beneficial in healthcare, public environments, and for users with mobility challenges. The project also includes backend integration using FastAPI and MongoDB, ensuring that every input submitted via the "Enter" key is securely stored for future reference. The modular structure of the project ensures ease of development, testing, and expansion. Moreover, the user-friendly interface, real-time feedback, and minimal hardware requirements make this system highly adaptable. Through this project, a strong foundation is laid for contactless human-computer interaction systems that prioritize convenience, accessibility, and hygiene. The overall system achieves its primary goals of functionality, usability, and real-time performance. It also offers a framework that can be customized for various specialized applications.

While the current system effectively enables contactless typing, there are several areas for potential improvement and expansion. Future enhancements could include the addition of multilingual support, allowing users to type in various languages based on regional preferences. Integration with voice recognition could offer users the option to switch between typing and speaking for more flexibility. Another promising feature is predictive text or autocorrect, which can improve typing speed and accuracy. The system could also be enhanced to work on mobile devices or tablets, broadening its usability across different platforms. Support for cloud-based data storage and syncing across devices can make it more scalable and user-friendly. Adding a login system with user profiles would allow individuals to track their typing history and access customized settings. Furthermore, improving gesture recognition using deep learning models could significantly increase input precision. Haptic feedback via vibrations or audio cues may also be explored to enhance interactivity. Overall, the project has great potential to evolve into a fully-featured, AI-driven input system suitable for a wide range of applications.

REFERENCES

CONCEPTUAL REFERENCES (RESEARCH & INNOVATION)

1. Human-Computer Interaction (HCI)

- Dix, A., Finlay, J., Abowd, G., & Beale, R. Human-Computer Interaction. Pearson Education, 2004. (Supports user-centric design in adaptive technologies.)

2. Touchless Interfaces

- Rekimoto, J. (2002). SmartSkin: An infrastructure for freehand manipulation on interactive surfaces. SIGCHI. (Describes gesture-based, non-contact interaction.)

3. Assistive Technology

- Cook, A. M., & Polgar, J. M. Assistive Technologies: Principles and Practice. Elsevier Health Sciences, 2014. (Relevant for your project's accessibility goals.)

TECHNICAL REFERENCES (TOOLS & LIBRARIES)

1. OpenCV Library

- Bradski, G. The OpenCV Library. Dr. Dobb's Journal of Software Tools, 2000. (Used for real-time hand tracking and gesture detection.)

2. MediaPipe by Google

- Lugaresi et al. (2019). MediaPipe: A Framework for Building Perception Pipelines. arXiv preprint arXiv:1906.08172. (Provides fast, cross-platform hand landmark detection.)

3. Python & FastAPI

- Sebastián Ramírez. FastAPI Documentation. <https://fastapi.tiangolo.com> (Backend for keyboard event handling or cloud integration.)

4. Tesseract OCR (if used for any text recognition)

- Smith, R. (2007). An Overview of the Tesseract OCR Engine. ICDAR.